KU LEUVEN

FACULTEIT INDUSTRIELE
INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Geavanceerde Computerarchitectuur

Report 4
MEI-ecfs

Charlotte BERT
Sander HAUSTRAETE

Academiejaar 2025 - 2026

1 Context

In dit labo wordt onderzocht hoe geheugenaccess en verschillende GPU-geheugentypes de prestaties van parallelle programma's beïnvloeden. In het eerste deel worden kernels ontwikkeld voor het kopiëren en transponeren van een matrix, waarbij het effect van coalesced memory access op de uitvoeringstijd wordt geanalyseerd. In het tweede deel wordt matrixvermenigvuldiging geïmplementeerd met verschillende geheugentypes (global, shared en constant memory). De uitvoeringstijden worden gemeten en vergeleken om inzicht te krijgen in de impact van geheugenbeheer op de efficiëntie van GPU-kernels

2 Methodologie

Eerst hebben we code geschreven om een matrix te kopiëren en te transponeren met de CPU. Vervolgens werden GPU-kernels geschreven die hetzelfde deden, en de resultaten hiervan werden gecontroleerd met behulp van de CPU-implementaties. Tot slot werden tijdsmetingen toegevoegd en de effectieve bandbreedte berekend, bij zowel de CPU- als GPU-implementaties, om te analyseren wat de invloed van coalesced of niet-coalesced memory access is, en dit te vergelijken met de metingen bij de CPU.

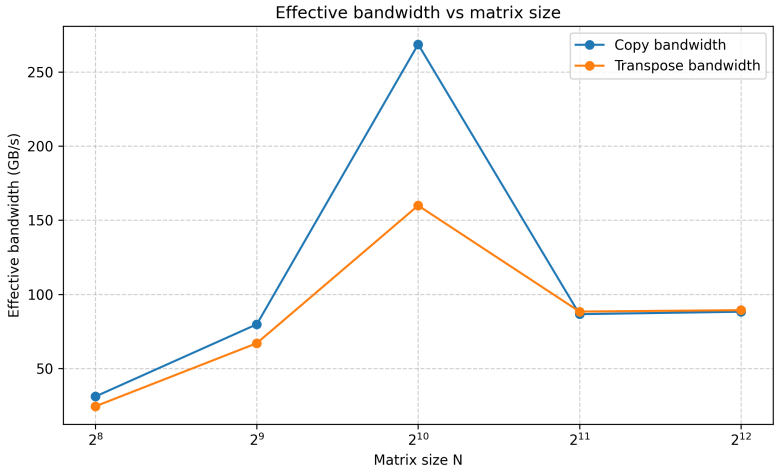
Voor het tweede deel van het labo zijn we begonnen met het ontwerpen van een kernel die 2 matrices vermenigvuldigt in global-only memory, en een CPU functie die hetzelfde doet. Hierna hebben we kernels geschreven die dit doen met gebruik van global en shared memory en van global en constant memory. Voor deze drie kernels hebben we de uitvoeringstijd met elkaar vergeleken en geanalyseerd.

3 Observatie/ resultaat

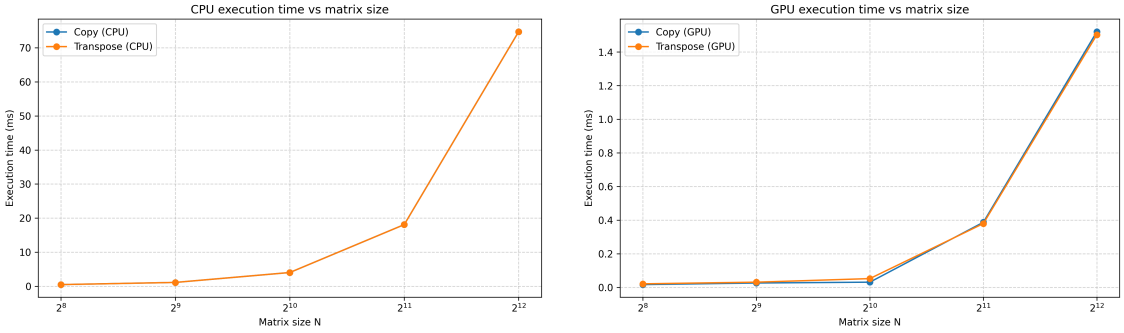
3.1 Deel 1: Coalesced memory access

```
Command: part_1
      N        op      cpu_ms      avg_ms      BW_GBs  ok?
    256      copy    0.473686   0.0168807     31.0585    Y
    256 transpose    0.473686   0.0214476      24.445    Y
    512      copy     1.10903   0.0262912     79.7662    Y
    512 transpose     1.10903   0.0312934     67.0158    Y
   1024      copy     4.01996   0.0312165     268.724    Y
   1024 transpose     4.01996   0.0524544     159.922    Y
   2048      copy     18.0891    0.387466     86.5996    Y
   2048 transpose     18.0891    0.379771     88.3544    Y
   4096      copy     74.7412     1.52124     88.2292    Y
   4096 transpose     74.7412     1.50288     89.3071    Y
```

Figuur 1: Terminal output



Figuur 2: Effectieve bandbreedte versus matrix grootte bij GPU kernels



(a) CPU uitvoeringstijd versus matrix grootte.

(b) GPU uitvoeringstijd versus matrix grootte.

Figuur 3: Vergelijking van CPU en GPU uitvoeringstijden.

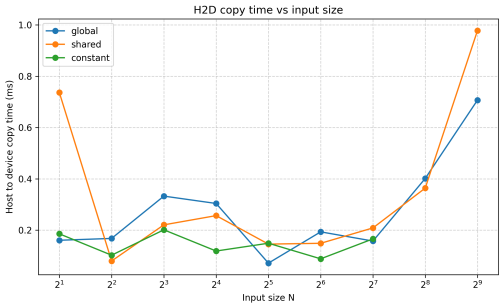
De prestaties veranderen naarmate de matrixgrootte toeneemt. Zelfs voor relatief kleine matrices is de GPU duidelijk sneller dan de CPU (zie figuur 3). Dit komt doordat de GPU een hoge mate van parallelisme biedt en efficiënter gebruikmaakt van de geheugenbandbreedte, waardoor de verwerking van matrices sneller uitgevoerd kunnen worden.

We zien dat de uitvoeringstijden van de copy en transpose kernels dicht bij elkaar liggen (figuur 3b), maar dat de effectieve bandbreedte van de copy kernel groter is dan die van de transpose kernel(figuur 2). Dit verschil is te verklaren doordat de copy kernel gebruik maakt van coalesced memory acces, terwijl de transpose kernel niet coalesced memory access gebruikt. Bij niet coalesced memory access worden de geheugenoperaties minder efficiënt gebundeld, waardoor er meerdere afzonderlijke geheugenoperaties nodig zijn. Dit verhoogt de geheugen latency en verlaagt dus de benutting van de beschikbare bandbreedte.

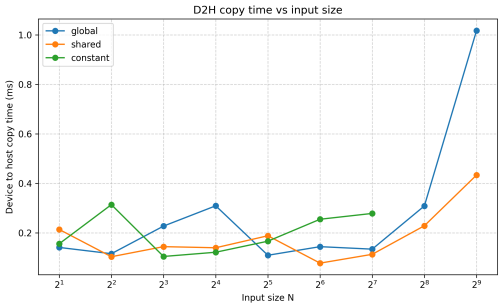
3.2 Deel 2: Global, shared, and constant memory

Hoeveel tijd kost geheugenmigratie (CPU naar GPU globaal; GPU globaal naar GPU gedeeld)?

De migratie van de CPU naar het globale GPU-geheugen (Host-to-Device) gebeurt door data naar het globale geheugen te kopiëren voordat de kernel wordt uitgevoerd. Zoals af te lezen in Figuur 4 is deze kopieertijd voor kleinere matrices ($N = 2^1$ tot 2^7) relatief constant en schommelt deze tussen de $0,1$ en $0,3$ ms. Zodra de invoer toeneemt tot $N = 2^9$, stijgt deze tijd naar circa $0,7$ tot $1,0$ ms. De migratie van globaal naar gedeeld (shared) geheugen verloopt anders, dit wordt per thread op de GPU zelf uitgevoerd tijdens de kernel run. Omdat deze datamigratie binnen de kernel plaatsvindt is deze niet direct als losse transfer te timen. Wel is de impact hiervan indirect zichtbaar in Figuur 6a. Het verschil van circa $0,3$ ms (bij $N = 2^9$) tussen de globale en de gedeelde kernel wordt grotendeels veroorzaakt door deze migratie-overhead en de verplichte synchronisatiestap.



Figuur 4: H2D kopiëertijd versus input grootte

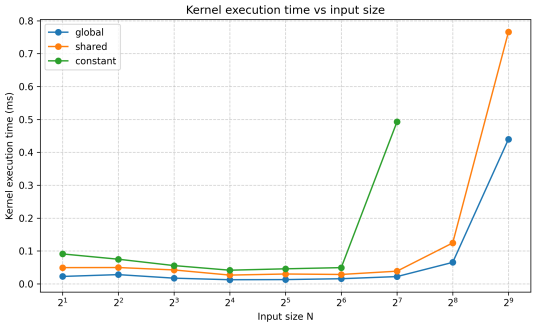


Figuur 5: D2H kopiëertijd versus input grootte

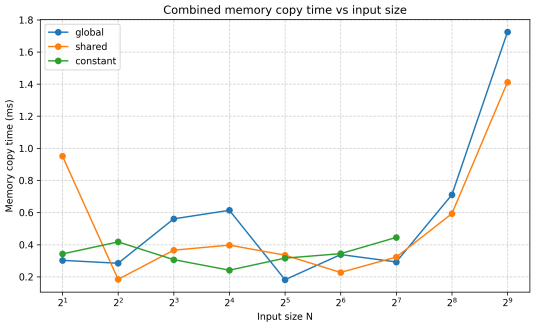
Hoe vergelijken de uitvoeringstijden van matrixvermenigvuldiging wanneer de drie verschillende geheugentypen worden gebruikt?

Uit de resultaten in Figuur 6a komen opvallende verschillen naar voren:

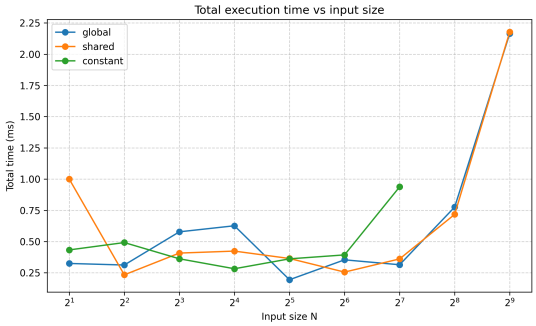
- **Globaal geheugen:** Dit geheugen bevindt zich off-device en heeft een hoge toegangslatency. Tegen de verwachting in blijkt deze kernel voor de geteste invoergroottes (tot $N = 2^9$) echter de snelste uitvoeringstijd op te leveren.
- **Gedeeld (shared) geheugen:** Dit geheugen bevindt zich on-device, heeft een lage toegangslatency en wordt gedeeld door de threads van één enkel blok. In theorie is dit sneller voor matrixvermenigvuldiging, maar de grafiek toont aan dat de kernel langzamer presteert dan de globale variant. De overhead van de datamigratie en draadsynchronisatie weegt voor deze matrixgroottes blijkbaar zwaarder dan de tijdswinst van de snellere geheugentoegang.
- **Constant geheugen:** Ook dit geheugen bevindt zich off-device, maar is specifiek geoptimaliseerd voor leesbewerkingen met een lage latency. Het presteert goed bij een kleine invoer, maar de curve stopt bij $N = 2^7$. Dit toont aan dat de theorie dat constant memory een



(a) Kernel uitvoeringstijd



(b) Gecombineerde kopiëertijd



(c) Totale uitvoeringstijd

Figuur 6: Uitsplitsing van de uitvoeringstijden versus input grootte (200 samples).

strikte hardwarelimiet heeft, wat doorgaans beperkt is tot 64 KB. Grotere matrices passen hier simpelweg niet in.

Hoe schalen de tijden met de invoer- en/of roostermaat?

Voor kleine invoergroottes ($N \leq 2^7$) blijven zowel de totale uitvoeringstijd als de kopieertijden redelijk constant. Hier is de zuivere rekentijd nog verwaarloosbaar en wordt de tijd gedomineerd door algemene overhead zoals de kernel launch latency. Bij een grotere invoer is er in alle figuren een duidelijke exponentiële stijging zichtbaar. In de pure globale implementatie moeten de rijen van matrix A en de kolommen van matrix B namelijk $N-1$ keer opnieuw uit het trage globale geheugen worden uitgelezen. Om dit schaalprobleem in theorie te verbeteren via gedeeld geheugen worden er vierkante blokken (tiles) gedefinieerd met grootte $BLOCK_SIZ$E. Elke thread kopieert hierbij 1 element van zowel A als B naar het gedeelde geheugen. Het voordeel van deze roosterverdeling is dat de data nog slechts k-keer herlezen hoeft te worden, waarbij $k = \frac{N}{BLOCK_SIZE}$. Hoe groter de matrix N wordt, hoe groter de winst van deze sub-blokken theoretisch zou moeten worden ten opzichte van de globale variant.

4 Conclusie

In dit labo is de cruciale rol van geheugentoegangspatronen en GPU-geheugentypes bij parallelle kernels geëvalueerd. Uit het eerste deel blijkt dat coalesced memory access essentieel is om de effectieve bandbreedte te maximaliseren. Het tweede deel toont aan dat het gebruik van sneller on-device shared memory niet automatisch tot kortere uitvoeringstijden leidt ten opzichte van global memory. Voor de geteste matrixgroottes woog de interne migratie- en synchronisatie-overhead in de praktijk namelijk zwaarder dan de behaalde rekenwinst. Verder bevestigden de metingen dat constant memory door zijn geoptimaliseerde leesbewerkingen zeer efficiënt is, maar door de strikte limiet van 64 KB snel vastloopt bij grotere datasets. Optimale GPU-prestaties hebben een doordachte afweging tussen datamigratie, de gekozen block size en het naleven van coalesced toegangspatronen nodig.

```cpp
#include <iostream>
#include <cuda_runtime.h>
#include <vector>
#include <chrono>
#include <iomanip>

#define BLOCK_SIZE 16 // voor shared memory

#define MAX_CONST_N 128

__constant__ int B_const[MAX_CONST_N * MAX_CONST_N];

// Kernels
// coalesced memory acces kernels
__global__ void copy_matrix(const int* A, int* B, int N) { //copy A to B
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        int idx = row * N + col;
        B[idx] = A[idx];
    }
}

__global__ void transpose_matrix(const int* A, int* B, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        int idx_in = row * N + col;
        int idx_out = col * N + row;

        B[idx_out] = A[idx_in];
    }
}

// global, shared and constant memory multiplication kernels (part 2)

//global
__global__ void multiply_global(const int *A, const int *B, int *C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        int sum = 0;
        for (int k=0; k < N; k++) {
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
```

```
}

//global and shared
__global__ void multiply_global_and_shared(const int *A, const int *B, int *C,
    int N) {
    __shared__ int As[BLOCK_SIZE][BLOCK_SIZE];
    __shared__ int Bs[BLOCK_SIZE][BLOCK_SIZE];

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    int sum = 0;

    int num_blocks = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;

    for (int i = 0; i < num_blocks; i++) {
        //global indices for shared memory
        int a_col = i * BLOCK_SIZE + threadIdx.x;
        int b_row = i * BLOCK_SIZE + threadIdx.y;

        //each thread copies one A and one B element to shared memory
        if (row < N && a_col < N) {
            As[threadIdx.y][threadIdx.x] = A[row * N + a_col];
        } else {
            As[threadIdx.y][threadIdx.x] = 0;
        }

        if (b_row < N && col < N) {
            Bs[threadIdx.y][threadIdx.x] = B[b_row * N + col];
        } else {
            Bs[threadIdx.y][threadIdx.x] = 0;
        }

        __syncthreads();

        for (int k = 0; k < BLOCK_SIZE; k++) {
            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        }

        __syncthreads();
    }

    if (row < N && col < N) {
        C[row * N + col] = sum;
    }
}

// global and constant
__global__ void multiply_global_and_constant(const int *A, int *C, int N) {
    //B in constant memory
    int row = blockIdx.y * blockDim.y + threadIdx.y;
```

```
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        int sum = 0;
        for (int k = 0; k < N; k++) {
            sum += A[row * N + k] * B_const[k * N + col];
        }
        C[row * N + col] = sum;
    }
}


// CPU Functions
static void print_matrix(const std::vector<int>& M, int N) {
    for (int r = 0; r < N; r++) {
        for (int c = 0; c < N; c++) {
            std::cout << M[r*N+c] << " ";
        }
        std::cout << std::endl;
    }
}

static bool check_if_same_matrix(const std::vector<int>& A, const std::vector<
    int>& B) {
    for (int i = 0; i < A.size(); i++) {
        if (A[i] != B[i]) {
            return false;
        }
    }
    return true;
}

void cpu_copy_matrix(const std::vector<int>& A, std::vector<int>& B, int N) {
    for (int r = 0; r < N; r++) {
        for (int c = 0 ; c < N; c++) {
            int idx = r * N + c;
            B[idx] = A[idx];
        }
    }
}

void cpu_transpose_matrix(const std::vector<int>& A, std::vector<int>& B, int N
    ) {
    for (int r = 0; r < N; r++) {
        for (int c = 0 ; c < N; c++) {
            int idx_in = r * N + c;
            int idx_out = c * N + r;
            B[idx_out] = A[idx_in];
        }
    }
}
```

```
150
151 void cpu_multiply(const std::vector<int>& A, const std::vector<int>& B, std::
        vector<int>& C, int N) {
152     for (int r = 0; r < N; r++) {
153         for (int c = 0; c < N; c++) {
154             int sum = 0;
155             for (int k = 0; k < N; k++) {
156                 sum += A[r*N+k] * B[k*N+c];
157             }
158             C[r*N+c] = sum;
159         }
160     }
161 }
162
163 int main(void) {
164     std::string command="part_1";
165     std::cout << "Command: " << command << std::endl;
166
167     cudaEvent_t start, stop;
168     cudaEventCreate(&start);
169     cudaEventCreate(&stop);
170
171     if (command=="copy_simple") {
172         int N = 2;
173
174         dim3 block(16, 16);   // 256 threads total
175         dim3 grid((N + block.x - 1) / block.x,
176                   (N + block.y - 1) / block.y);
177
178         std::vector<int> A = {
179             0, 1,
180             0, 1
181         };
182
183         std::vector<int> B(N*N, 0);
184         std::vector<int> C(N*N, 0);
185
186         std::cout << "Matrix A:" << std::endl;
187         print_matrix(A, N);
188
189         const auto start_cpu = std::chrono::steady_clock::now();
190
191         cpu_copy_matrix(A, B, N);
192
193         const auto end_cpu = std::chrono::steady_clock::now();
194         const std::chrono::duration<double, std::milli> elapsed_cpu{end_cpu -
        start_cpu};
195
196         std::cout << "Matrix B (CPU):" << std::endl;
197         print_matrix(B, N);
198         std::cout << "Time taken: " << elapsed_cpu.count() << std::endl;
```

```
199
200         int *d_A, *d_C;
201
202         cudaMalloc((void**)&d_A, N*N * sizeof(int));
203         cudaMalloc((void**)&d_C, N*N * sizeof(int));
204
205         cudaMemcpy(d_A, A.data(), N*N * sizeof(int), cudaMemcpyHostToDevice);
206
207         cudaEventRecord(start);
208
209         copy_matrix<<<grid,block>>>(d_A, d_C, N);
210
211         cudaDeviceSynchronize();
212         cudaEventRecord(stop);
213         cudaEventSynchronize(stop);
214         float ms;
215         cudaEventElapsedTime(&ms, start, stop);
216
217         cudaMemcpy(C.data(), d_C, N*N * sizeof(int), cudaMemcpyDeviceToHost);
218
219         std::cout << "Matrix C (GPU):" << std::endl;
220         print_matrix(C, N);
221         std::cout << "Time taken: " << ms << std::endl;
222
223         cudaFree(d_A);
224         cudaFree(d_C);
225         std::cout << "B and C same matrix? " << (check_if_same_matrix(B, C) ? "
    True":"False") << std::endl;
226     } else if (command=="transpose_simple") {
227         int N = 2;
228
229         dim3 block(16, 16);   // 256 threads total
230         dim3 grid((N + block.x - 1) / block.x,
231                   (N + block.y - 1) / block.y);
232
233
234         std::vector<int> A = {
235             0, 1,
236             0, 1
237         };
238
239         std::vector<int> B(N*N, 0);
240         std::vector<int> C(N*N, 0);
241
242         std::cout << "Matrix A:" << std::endl;
243         print_matrix(A, N);
244
245         const auto start_cpu = std::chrono::steady_clock::now();
246
247         cpu_transpose_matrix(A, B, N);
248
```

```
249         const auto end_cpu = std::chrono::steady_clock::now();
250         const std::chrono::duration<double, std::milli> elapsed_cpu{end_cpu -
    start_cpu};
251 
252         std::cout << "Matrix B (CPU):" << std::endl;
253         print_matrix(B, N);
254         std::cout << "Time taken: " << elapsed_cpu.count() << std::endl;
255 
256         int *d_A, *d_C;
257 
258         cudaMalloc((void**)&d_A, N*N * sizeof(int));
259         cudaMalloc((void**)&d_C, N*N * sizeof(int));
260 
261         cudaMemcpy(d_A, A.data(), N*N * sizeof(int), cudaMemcpyHostToDevice);
262 
263         cudaEventRecord(start);
264 
265         transpose_matrix<<<grid,block>>>(d_A, d_C, N);
266 
267         cudaDeviceSynchronize();
268         cudaEventRecord(stop);
269         cudaEventSynchronize(stop);
270         float ms;
271         cudaEventElapsedTime(&ms, start, stop);
272 
273         cudaMemcpy(C.data(), d_C, N*N * sizeof(int), cudaMemcpyDeviceToHost);
274 
275         std::cout << "Matrix C (GPU):" << std::endl;
276         print_matrix(C, N);
277         std::cout << "Time taken: " << ms << std::endl;
278 
279         cudaFree(d_A);
280         cudaFree(d_C);
281         std::cout << "B and C same matrix? " << (check_if_same_matrix(B, C) ? "
    True":"False") << std::endl;
282     } else if (command=="part_1") {
283         int N_sizes[] = {256, 512, 1024, 2048, 4096};
284 
285         const int warmup_iterations = 10;
286         const int iterations = 200;
287 
288         std::cout
289         << std::setw(8)  << "N"
290         << std::setw(10) << "op"
291         << std::setw(12) << "cpu_ms"
292         << std::setw(12) << "avg_ms"
293         << std::setw(12) << "BW_GBs"
294         << std::setw(6)  << "ok?"
295         << std::endl;
296 
297         for (int N: N_sizes) {
```

```
298             std::vector<int> A(N*N, 0);
299             std::vector<int> B(N*N, 0);
300             std::vector<int> C(N*N, 0);
301 
302             for (int i = 0; i < N*N; i++) {
303                 A[i] = i;
304             }
305 
306             int *d_A, *d_C;
307             cudaMalloc((void**)&d_A, N*N * sizeof(int));
308             cudaMalloc((void**)&d_C, N*N * sizeof(int));
309 
310             cudaMemcpy(d_A, A.data(), N*N*sizeof(int), cudaMemcpyHostToDevice);
311             dim3 block(16, 16);   // 256 threads total
312             dim3 grid((N + block.x - 1) / block.x,
313                       (N + block.y - 1) / block.y);
314 
315 
316             // Copy
317             // CPU-implementation
318             const auto start_cpu = std::chrono::steady_clock::now();
319 
320             cpu_copy_matrix(A, B, N);
321 
322             const auto end_cpu = std::chrono::steady_clock::now();
323             const std::chrono::duration<double, std::milli> elapsed_cpu{end_cpu
    - start_cpu};
324             float ms_cpu_copy = elapsed_cpu.count();
325 
326             // GPU-implementation
327             // warmup
328             for (int i = 0; i < warmup_iterations; i++) {
329                 copy_matrix<<<grid, block>>>(d_A, d_C, N);
330             }
331             cudaDeviceSynchronize();
332 
333             // loop
334             float ms_total_copy = 0.0f;
335             for (int i = 0; i < iterations; i++) {
336                 float ms;
337                 cudaEventRecord(start);
338                 copy_matrix<<<grid, block>>>(d_A, d_C, N);
339                 cudaEventRecord(stop);
340                 cudaEventSynchronize(stop);
341                 cudaEventElapsedTime(&ms, start, stop);
342                 ms_total_copy += ms;
343             }
344             float ms_avg_copy = ms_total_copy / iterations;
345 
346             cudaMemcpy(C.data(), d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);
347             bool ok_copy = check_if_same_matrix(B, C);
```

```
348
349             // BW
350             float bytes_moved_copy = 2 * N*N*sizeof(int);
351             float bw_copy = (bytes_moved_copy/1e9) / (ms_avg_copy * 1e-3);
352
353             std::cout
354             << std::setw(8)  << N
355             << std::setw(10) << "copy"
356             << std::setw(12) << ms_cpu_copy
357             << std::setw(12) << ms_avg_copy
358             << std::setw(12) << bw_copy
359             << std::setw(6)  << (ok_copy ? "Y" : "N")
360             << std::endl;
361
362
363             // Transpose
364             // CPU-implementation
365             const auto start_cpu_transpose = std::chrono::steady_clock::now();
366
367             cpu_transpose_matrix(A, B, N);
368
369             const auto end_cpu_transpose = std::chrono::steady_clock::now();
370             const std::chrono::duration<double, std::milli>
    elapsed_cpu_transpose{end_cpu - start_cpu};
371             float ms_cpu_transpose = elapsed_cpu_transpose.count();
372
373             // GPU-implementation
374             // warmup
375             for (int i = 0; i < warmup_iterations; i++) {
376                 transpose_matrix<<<grid, block>>>(d_A, d_C, N);
377             }
378             cudaDeviceSynchronize();
379
380             // loop
381             float ms_total_transpose = 0.0f;
382             for (int i = 0; i < iterations; i++) {
383                 float ms;
384                 cudaEventRecord(start);
385                 transpose_matrix<<<grid, block>>>(d_A, d_C, N);
386                 cudaEventRecord(stop);
387                 cudaEventSynchronize(stop);
388                 cudaEventElapsedTime(&ms, start, stop);
389                 ms_total_transpose += ms;
390             }
391             float ms_avg_transpose = ms_total_transpose / iterations;
392
393             cudaMemcpy(C.data(), d_C, N*N*sizeof(int), cudaMemcpyDeviceToHost);
394             bool ok_transpose = check_if_same_matrix(B, C);
395
396             // BW
397             float bytes_moved_transpose = 2 * N*N*sizeof(int);
```

```
            float bw_transpose = (bytes_moved_transpose/1e9) / (ms_avg_transpose * 1e-3);

            std::cout
            << std::setw(8)  << N
            << std::setw(10) << "transpose"
            << std::setw(12) << ms_cpu_transpose
            << std::setw(12) << ms_avg_transpose
            << std::setw(12) << bw_transpose
            << std::setw(6)  << (ok_transpose ? "Y" : "N")
            << std::endl;

            cudaFree(d_A);
            cudaFree(d_C);
        }
    } else if (command=="multiply_global") {
        int N = 2;

        dim3 block(16, 16);   // 256 threads total
        dim3 grid((N + block.x - 1) / block.x,
                  (N + block.y - 1) / block.y);


        std::vector<int> A = {
            0, 1,
            0, 1
        };

        std::vector<int> B = {
            1, 0,
            1, 0
        };

        std::vector<int> C(N*N, 0);

        std::cout << "Matrix A:" << std::endl;
        print_matrix(A, N);

        std::cout << "Matrix B:" << std::endl;
        print_matrix(B, N);

        int *d_A, *d_B, *d_C;

        cudaMalloc((void**)&d_A, N*N * sizeof(int));
        cudaMalloc((void**)&d_B, N*N * sizeof(int));
        cudaMalloc((void**)&d_C, N*N * sizeof(int));

        cudaMemcpy(d_A, A.data(), N*N * sizeof(int), cudaMemcpyHostToDevice);
        cudaMemcpy(d_B, B.data(), N*N * sizeof(int), cudaMemcpyHostToDevice);

        cudaEventRecord(start);
```

```
448
449         multiply_global<<<grid,block>>>(d_A, d_B, d_C, N);
450
451         cudaDeviceSynchronize();
452         cudaEventRecord(stop);
453         cudaEventSynchronize(stop);
454         float ms;
455         cudaEventElapsedTime(&ms, start, stop);
456
457         cudaMemcpy(C.data(), d_C, N*N * sizeof(int), cudaMemcpyDeviceToHost);
458
459         std::cout << "Matrix C = A*B (global):" << std::endl;
460         print_matrix(C, N);
461         std::cout << "Time taken: " << ms << std::endl;
462
463         cudaFree(d_A);
464         cudaFree(d_B);
465         cudaFree(d_C);
466     } else if (command=="multiply_shared") {
467         int N = 2;
468
469         dim3 block(BLOCK_SIZE, BLOCK_SIZE);   // 256 threads total
470         dim3 grid((N + block.x - 1) / block.x,
471                   (N + block.y - 1) / block.y);
472
473
474         std::vector<int> A = {
475             0, 1,
476             0, 1
477         };
478
479         std::vector<int> B = {
480             1, 0,
481             1, 0
482         };
483
484         std::vector<int> C(N*N, 0);
485
486         std::cout << "Matrix A:" << std::endl;
487         print_matrix(A, N);
488
489         std::cout << "Matrix B:" << std::endl;
490         print_matrix(B, N);
491
492         int *d_A, *d_B, *d_C;
493
494         cudaMalloc((void**)&d_A, N*N * sizeof(int));
495         cudaMalloc((void**)&d_B, N*N * sizeof(int));
496         cudaMalloc((void**)&d_C, N*N * sizeof(int));
497
498         cudaMemcpy(d_A, A.data(), N*N * sizeof(int), cudaMemcpyHostToDevice);
```

```
        cudaMemcpy(d_B, B.data(), N*N * sizeof(int), cudaMemcpyHostToDevice);

        cudaEventRecord(start);

        multiply_global_and_shared<<<grid,block>>>(d_A, d_B, d_C, N);

        cudaDeviceSynchronize();
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        float ms;
        cudaEventElapsedTime(&ms, start, stop);

        cudaMemcpy(C.data(), d_C, N*N * sizeof(int), cudaMemcpyDeviceToHost);

        std::cout << "Matrix C = A*B (global and shared):" << std::endl;
        print_matrix(C, N);
        std::cout << "Time taken: " << ms << std::endl;

        cudaFree(d_A);
        cudaFree(d_B);
        cudaFree(d_C);
    } else if (command=="multiply_constant") {
        int N = 2;

        dim3 block(16, 16);   // 256 threads total
        dim3 grid((N + block.x - 1) / block.x,
                  (N + block.y - 1) / block.y);


        std::vector<int> A = {
            0, 1,
            0, 1
        };

        std::vector<int> B = {
            1, 0,
            1, 0
        };

        std::vector<int> C(N*N, 0);

        std::cout << "Matrix A:" << std::endl;
        print_matrix(A, N);

        std::cout << "Matrix B:" << std::endl;
        print_matrix(B, N);

        int *d_A, *d_B, *d_C;

        cudaMalloc((void**)&d_A, N*N * sizeof(int));
        cudaMalloc((void**)&d_B, N*N * sizeof(int));
```

```
        cudaMalloc((void**)&d_C, N*N * sizeof(int));

        cudaMemcpy(d_A, A.data(), N*N * sizeof(int), cudaMemcpyHostToDevice);
        cudaMemcpy(d_B, B.data(), N*N * sizeof(int), cudaMemcpyHostToDevice);

        cudaEventRecord(start);

        if (N <= MAX_CONST_N) {
            cudaMemcpyToSymbol(B_const, B.data(), N*N*sizeof(int));
            multiply_global_and_constant<<<grid, block>>>(d_A, d_C, N);
        }

        cudaDeviceSynchronize();
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        float ms;
        cudaEventElapsedTime(&ms, start, stop);

        cudaMemcpy(C.data(), d_C, N*N * sizeof(int), cudaMemcpyDeviceToHost);

        std::cout << "Matrix C = A*B (global and constant):" << std::endl;
        print_matrix(C, N);
        std::cout << "Time taken: " << ms << std::endl;

        cudaFree(d_A);
        cudaFree(d_B);
        cudaFree(d_C);
    } else if (command=="part_2") {
        int N_sizes[] = {2, 4, 8, 16, 32, 64, 128, 256, 512};

        const int warmup_iterations = 10;
        const int iterations = 200;

        std::cout << std::fixed << std::setprecision(6);
        std::cout
        << std::setw(8)  << "N"
        << std::setw(14) << "kernel"
        << std::setw(14) << "H2D_ms"
        << std::setw(14) << "kernel_ms"
        << std::setw(14) << "D2H_ms"
        << std::setw(10) << "correct"
        << std::endl;

        for (int N: N_sizes) {
            std::vector<int> A(N*N, 0);
            std::vector<int> B(N*N, 0);
            std::vector<int> C(N*N, 0);
            std::vector<int> C_cpu(N*N, 0);

            // fill matrices
            for (int i = 0; i < N*N; i++) {
```

```
601                 A[i] = i;
602                 B[i] = i;
603             }
604 
605             cpu_multiply(A, B, C_cpu, N);
606 
607             int *d_A, *d_B, *d_C;
608             cudaMalloc((void**)&d_A, N*N * sizeof(int));
609             cudaMalloc((void**)&d_B, N*N * sizeof(int));
610             cudaMalloc((void**)&d_C, N*N * sizeof(int));
611 
612             dim3 block(BLOCK_SIZE, BLOCK_SIZE);
613             dim3 grid((N + block.x - 1) / block.x,
614                     (N + block.y - 1) / block.y);
615 
616             float ms_h2d = 0.0f;
617             float ms_d2h = 0.0f;
618             float ms_kernel_total = 0.0f;
619             float ms_kernel_avg = 0.0f;
620 
621             // global kernel
622             // warmup memcpy
623             cudaMemcpy(d_A, A.data(), N*N * sizeof(int), cudaMemcpyHostToDevice
    );
624 
625             cudaEventRecord(start);
626             cudaMemcpy(d_A, A.data(), N*N * sizeof(int), cudaMemcpyHostToDevice
    );
627             cudaMemcpy(d_B, B.data(), N*N * sizeof(int), cudaMemcpyHostToDevice
    );
628             cudaEventRecord(stop);
629             cudaEventSynchronize(stop);
630             cudaEventElapsedTime(&ms_h2d, start, stop);
631 
632             for (int i = 0; i < warmup_iterations; i++) {
633                 multiply_global<<<grid, block>>>(d_A, d_B, d_C, N);
634             }
635             cudaDeviceSynchronize();
636 
637             for (int i = 0; i < iterations; i++) {
638                 float ms;
639                 cudaEventRecord(start);
640                 multiply_global<<<grid, block>>>(d_A, d_B, d_C, N);
641                 cudaEventRecord(stop);
642                 cudaEventSynchronize(stop);
643                 cudaEventElapsedTime(&ms, start, stop);
644                 ms_kernel_total += ms;
645             }
646             ms_kernel_avg = ms_kernel_total / iterations;
647 
648             cudaEventRecord(start);
```

```
649             cudaMemcpy(C.data(), d_C, N*N * sizeof(int), cudaMemcpyDeviceToHost
    );
650             cudaEventRecord(stop);
651             cudaEventSynchronize(stop);
652             cudaEventElapsedTime(&ms_d2h, start, stop);
653 
654             std::cout
655                 << std::setw(8)  << N
656                 << std::setw(14) << "global"
657                 << std::setw(14) << ms_h2d
658                 << std::setw(14) << ms_kernel_avg
659                 << std::setw(14) << ms_d2h
660                 << std::setw(10) << (check_if_same_matrix(C, C_cpu) ? "Y" : "N"
    )
661                 << std::endl;
662 
663             // Shared
664 
665             cudaEventRecord(start);
666             cudaMemcpy(d_A, A.data(), N*N * sizeof(int), cudaMemcpyHostToDevice
    );
667             cudaMemcpy(d_B, B.data(), N*N * sizeof(int), cudaMemcpyHostToDevice
    );
668             cudaEventRecord(stop);
669             cudaEventSynchronize(stop);
670             cudaEventElapsedTime(&ms_h2d, start, stop);
671 
672             for (int i = 0; i < warmup_iterations; i++) {
673                 multiply_global_and_shared<<<grid, block>>>(d_A, d_B, d_C, N);
674             }
675             cudaDeviceSynchronize();
676 
677             for (int i = 0; i < iterations; i++) {
678                 float ms;
679                 cudaEventRecord(start);
680                 multiply_global_and_shared<<<grid, block>>>(d_A, d_B, d_C, N);
681                 cudaEventRecord(stop);
682                 cudaEventSynchronize(stop);
683                 cudaEventElapsedTime(&ms, start, stop);
684                 ms_kernel_total += ms;
685             }
686             ms_kernel_avg = ms_kernel_total / iterations;
687 
688             cudaEventRecord(start);
689             cudaMemcpy(C.data(), d_C, N*N * sizeof(int), cudaMemcpyDeviceToHost
    );
690             cudaEventRecord(stop);
691             cudaEventSynchronize(stop);
692             cudaEventElapsedTime(&ms_d2h, start, stop);
693 
694             std::cout
```

```
                << std::setw(8)  << N
                << std::setw(14) << "shared"
                << std::setw(14) << ms_h2d
                << std::setw(14) << ms_kernel_avg
                << std::setw(14) << ms_d2h
                << std::setw(10) << (check_if_same_matrix(C, C_cpu) ? "Y" : "N")
                << std::endl;

            // Constant

            if (N <= MAX_CONST_N) {
                cudaEventRecord(start);
                cudaMemcpy(d_A, A.data(), N*N * sizeof(int), cudaMemcpyHostToDevice);
                cudaMemcpyToSymbol(B_const, B.data(), N*N * sizeof(int));
                cudaEventRecord(stop);
                cudaEventSynchronize(stop);
                cudaEventElapsedTime(&ms_h2d, start, stop);

                for (int i = 0; i < warmup_iterations; i++) {
                    multiply_global_and_constant<<<grid, block>>>(d_A, d_C, N);
                }
                cudaDeviceSynchronize();

                for (int i = 0; i < iterations; i++) {
                    float ms;
                    cudaEventRecord(start);
                    multiply_global_and_constant<<<grid, block>>>(d_A, d_C, N);
                    cudaEventRecord(stop);
                    cudaEventSynchronize(stop);
                    cudaEventElapsedTime(&ms, start, stop);
                    ms_kernel_total += ms;
                }
                ms_kernel_avg = ms_kernel_total / iterations;

                cudaEventRecord(start);
                cudaMemcpy(C.data(), d_C, N*N * sizeof(int), cudaMemcpyDeviceToHost);
                cudaEventRecord(stop);
                cudaEventSynchronize(stop);
                cudaEventElapsedTime(&ms_d2h, start, stop);

                std::cout
                    << std::setw(8)  << N
                    << std::setw(14) << "constant"
                    << std::setw(14) << ms_h2d
                    << std::setw(14) << ms_kernel_avg
                    << std::setw(14) << ms_d2h
                    << std::setw(10) << (check_if_same_matrix(C, C_cpu) ? "Y" : "N")
```

```
742                     << std::endl;
743             }
744
745             cudaFree(d_A);
746             cudaFree(d_B);
747             cudaFree(d_C);
748         }
749     }
750
751     cudaEventDestroy(start);
752     cudaEventDestroy(stop);
753 }
```

Codefragment 1: Code labo 4

FACULTEIT INDUSTRIELE INGENIEURSWETENSCHAPPEN
TECHNOLOGIECAMPUS GENT
Gebroeders De Smetstraat 1
9000 GENT, België
tel. + 32 92 65 86 10
fax + 32 92 25 62 69
iiw.gent@kuleuven.be
www.iiw.kuleuven.be

LID VAN ASSOCIATIE KU LEUVEN